



slington college
(इस्लिङ्टन कलेज)

Module Code & Module Title

CS6P05NI Project

Assessment Weightage & Type

5% FYP Proposal

Semester

2023 Autumn

Network Vulnerability Assessment Tool

Student Name: Pawan Shah

London Met ID: 21040616

College ID: np01nt4a210010

Internal Supervisor: Akchayat Bikram Joshi

External Supervisor: Samir Gautam

Assignment Due Date: 29th November 2023

Assignment Submission Date: 29th November 2023

Word Count (Where Required): 1995

I confirm that I understand my coursework needs to be submitted online via Google Classroom under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table of Contents

1	Introduction	1
1.1	Problem Scenario.....	1
1.2	Project as a Solution	2
2	Aim and Objectives	3
3	Expected Outcomes and Deliverables	4
4	Project Risks, Threats, and Contingency Plan	5
5	Methodology	6
5.1	Flowchart.....	10
5.2	System architecture diagram.....	11
6	Resource Requirement	12
7	Work Breakdown Structure (WBS).....	13
8	Milestone.....	14
9	Project Gantt Chart	15
10	Conclusion.....	16
11	References	17
12	Appendix	18
12.1	Pros and Cons of the Evolutionary Prototyping Model.....	18

Table of Figures

Figure 1: System architecture diagram (Lapada, 2019)	6
Figure 2: Flowchart of NVA tool	10
Figure 3: System architecture diagram.....	11
Figure 4: Work breakdown structure (WBS)	13
Figure 5: Milestone	14
Figure 6: Gantt Chart.....	15

1 Introduction

In today's interconnected world, where organizations rely heavily on digital infrastructure and the Internet, the security of computer networks has become a paramount concern (Team, 2023). The ever-evolving landscape of cyber threats poses a significant challenge to businesses, governments, and individuals. To effectively safeguard against these threats, it is essential to proactively identify and address vulnerabilities within network systems. This is where a network vulnerability assessment (NVA) tool comes into play.

A network vulnerability assessment tool is crucial to any organization's cybersecurity strategy. It is a software solution that serves to systematically identify, evaluate, and prioritize potential weaknesses within a network's architecture and its connected devices. Doing so empowers administrators and security professionals to take proactive measures in mitigating vulnerabilities before malicious actors can exploit them.

1.1 Problem Scenario

The organization is faced with several problems that the network vulnerability assessment (NVA) tool aims to address. The problems are given below:

- Lack of awareness regarding all devices connected to the network, leading to undiscovered hosts. This lack of visibility can result in unmonitored entry points, making the network vulnerable to unauthorized access and potential security breaches (Murray, 2023).
- Unsecured ports and services, highlighting the need for a systematic approach to identify open ports and the associated services running on them. Without this crucial information, the organization risks overlooking potential vulnerabilities that could be exploited by attackers.
- Lack of malicious URL detection within the network.

1.2 Project as a Solution

The Network Vulnerability Assessment Tool is designed to address the problems outlined in the problem scenario section, thereby enhancing network security. The tool scans the network to discover all active hosts, providing visibility into all devices connected to the network. This helps prevent unmonitored entry points and reduces the risk of unauthorized access and potential security breaches. The tool performs comprehensive port scanning and detects the service and its version running on each open port. This systematic approach helps identify potential vulnerabilities that could be exploited by attackers. The tool integrates with a robust vulnerability database to match the identified services and their versions to known vulnerabilities. This proactive approach helps the organization stay ahead of potential threats by identifying and addressing vulnerabilities before they can be exploited. The tool uses the VirusTotal API to detect malicious URLs within the network traffic. This adds an extra layer of defense against potential cyber threats.

2 Aim and Objectives

The project's aim is to develop a comprehensive network vulnerability assessment (NVA) tool that empowers businesses to proactively detect and mitigate vulnerabilities within their network systems. It also enhances the overall security posture of networks, reducing the risk of cyber threats and fortifying the integrity, confidentiality, and availability of digital infrastructure.

The Objectives of the project are:

- Develop a powerful and user-friendly network vulnerability assessment (NVA) tool.
- Provide cost-effective solutions to the organization.
- Develop a standardized process for conducting in-depth vulnerability assessments.
- Develop a reporting mechanism that generates clear, concise, and actionable reports.
- Identify malicious links within a network.

3 Expected Outcomes and Deliverables

The expected outcomes of the network vulnerability assessment (NVA) tool encompass a robust set of features designed to fortify network security. The tool will conduct thorough scans to discover active hosts within the network and perform comprehensive port scanning to identify open ports and detect their respective services and service versions. Leveraging a robust vulnerability database, the NVA tool will match the identified services and service versions to potential vulnerabilities, offering an insightful assessment of the network's security posture. The tool's functionality extends to the generation of reports, providing network administrators and security professionals with actionable insights for mitigation. Additionally, the NVA tool will enhance its capabilities by incorporating the ability to detect malicious links on the network using the VirusTotal API, adding an extra layer of defense against potential cyber threats.

The deliverables will include the complete and well-documented source code, comprehensive documentation detailing the tool's architecture and functionality, detailed test reports, and a final report summarizing the project's development process, findings, and recommendations for future enhancements.

4 Project Risks, Threats, and Contingency Plan

The project risks and threats are given below:

- Inaccurate Scanning: The tool may not always accurately identify hosts, ports, or services due to network complexities or security measures in place.
- Database Dependence: The tool's effectiveness is heavily dependent on the vulnerability database. If the database is not updated regularly, the tool may miss new vulnerabilities.
- False Positives/Negatives: There's a risk of false positives where harmless services are flagged as vulnerabilities and false negatives where actual vulnerabilities are missed.
- API Limitations: The virustotal API used for detecting malicious URLs may have usage limitations.
- Programming language library and socket programming might face issues.

The Contingency Plans are given below:

- Monitor the usage of the virustotal API and ensure it stays within the allowed limits. Consider subscribing to a premium plan or using alternative APIs if necessary.
- Implement advanced scanning algorithms and techniques to improve the accuracy of host, port, and service detection.
- Improve the tool's algorithms to reduce false positives and negatives.
- Using an alternative programming language library for availability.

5 Methodology

For the development of this project, the selected methodology is the evolutionary prototyping model. This decision is rooted in the recognition of the need for a dynamic and adaptable approach to software development. Evolutionary prototyping aligns well with the project's goals by offering a flexible framework that encourages continuous refinement and enhancement throughout the development lifecycle. By using this method, we aim to address the inherent challenges in the project's scope and requirements.

The pros and cons are on the [appendix](#) section.

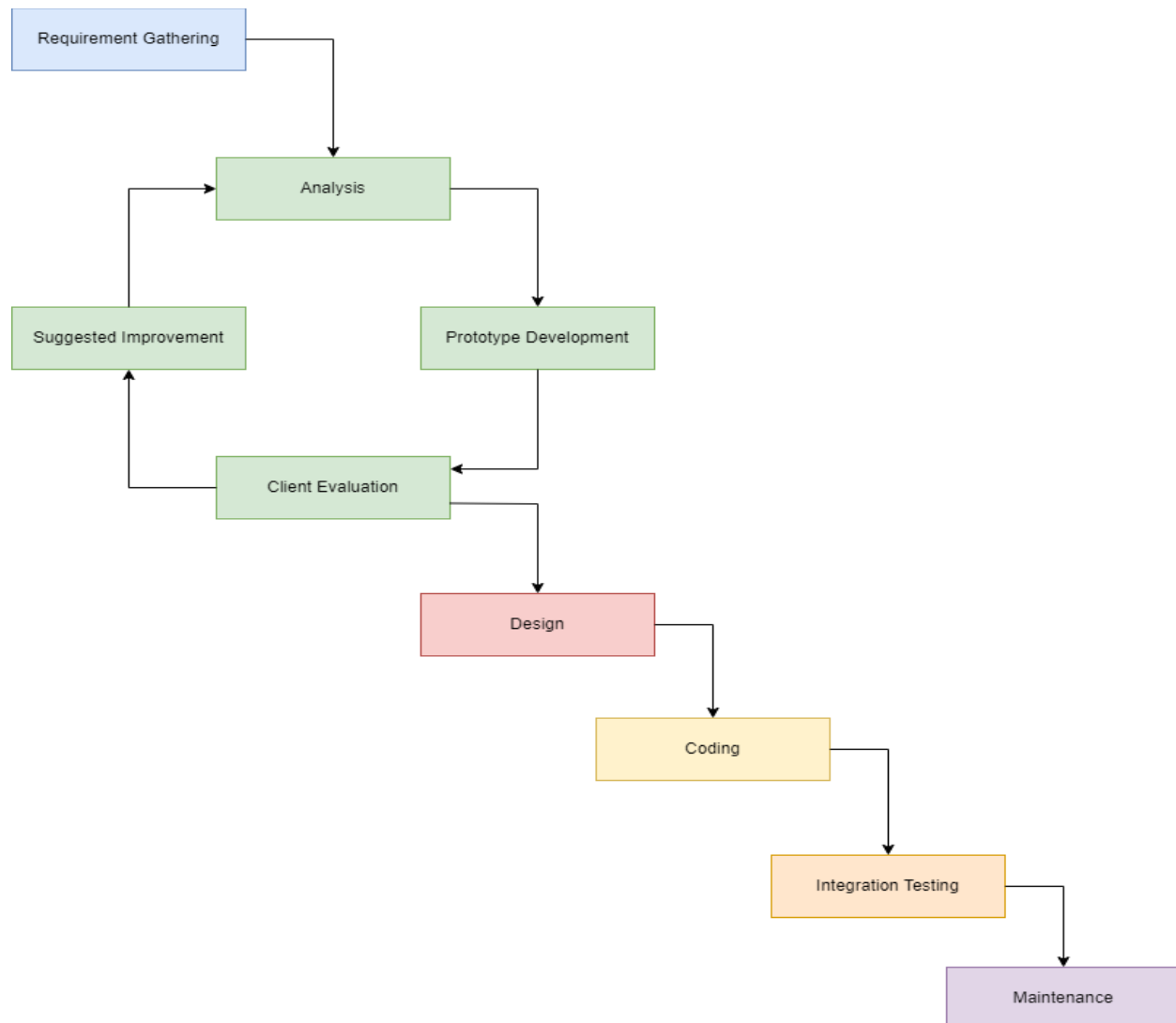


Figure 1: System architecture diagram (Lapada, 2019)

- **Requirement gathering**

The process begins with an initial understanding of the project's requirements. This crucial phase involves research, planning, and close collaboration between the development team and stakeholders to establish a foundational comprehension of the project's objectives and specifications (Carter, et al., 2001).

- **Analysis**

In the analysis phase, the focus is on gaining a comprehensive understanding of the project's initial requirements and laying the groundwork for the iterative development process. This phase involves close collaboration between the development team and stakeholders to gather, document, and analyze the core functionalities and specifications of the software (Carter, et al., 2001).

- **Prototype development**

In this phase, a basic prototype is developed to visualize and demonstrate the preliminary functionalities of the project. This prototype serves as a tangible representation of the envisioned software, allowing stakeholders to visualize and interact with key functionalities. The emphasis is on creating a functional model that captures the essence of the final product (BtechVibes, 2023).

- **Client evaluation**

The developed prototype undergoes rigorous assessment and feedback from the clients or end-users. This crucial step involves close collaboration and communication between the development team and stakeholders. Clients evaluate the prototype's functionalities, user interface, and overall alignment with their requirements. The iterative nature of this phase allows for continuous refinement based on client feedback, ensuring that the evolving software product remains closely tailored to the client's needs (Carter, et al., 2001).

- **Suggested Improvement**

In this phase, feedback from clients and stakeholders is analyzed, and necessary enhancements are identified. This phase focuses on refining and augmenting the prototype based on the insights gained during the client evaluation. It involves a systematic process of incorporating suggested improvements to ensure that the evolving software aligns more closely with user expectations and project goals. The emphasis is on continuous enhancement, allowing the development team to iteratively improve the software's features, functionality, and overall performance (Carter, et al., 2001).

- **Design**

The refined prototype serves as the foundation for creating a more comprehensive and detailed system design. Building upon the insights gained during previous iterations, the development team focuses on structuring the software architecture, defining system components, and specifying their interactions. This phase involves making critical design decisions to address the evolving requirements and enhancements identified in earlier stages. The goal is to create a robust and scalable design that guides the subsequent development phases, ensuring that the software continues to evolve in a structured and purposeful manner (Carter, et al., 2001).

- **Coding/ Development**

This phase involves implementing the design and functionality of the prototype based on the requirements gathered in the previous stages. During this phase, developers write the code that brings the prototype to life, transforming the design and functional specifications into a working model. This is where the prototype starts to take shape and the developers can start to see how the prototype will work in real life (Carter, et al., 2001).

- **Integration Testing**

In this phase, the prototype undergoes rigorous testing to identify any issues that may arise when the individual components interact with each other. This includes testing the data flow between different components, checking the coordination of different functions, and ensuring that the system works seamlessly as a whole. The goal of the Integration Testing phase is to detect and fix any integration errors before the system goes into the finalization stage. This helps ensure that the final product is reliable, meets user requirements, and provides a smooth user experience (Carter, et al., 2001).

- **Maintenance**

In the final phase, the focus shifts to sustaining the software's performance, addressing issues, and incorporating updates. This phase involves ongoing monitoring of the tool's functionality, responsiveness to emerging threats, and adapting to evolving network environments. Developers collaborate to identify and fix any potential issues, ensuring the tool remains effective in identifying vulnerabilities (Carter, et al., 2001).

5.1 Flowchart

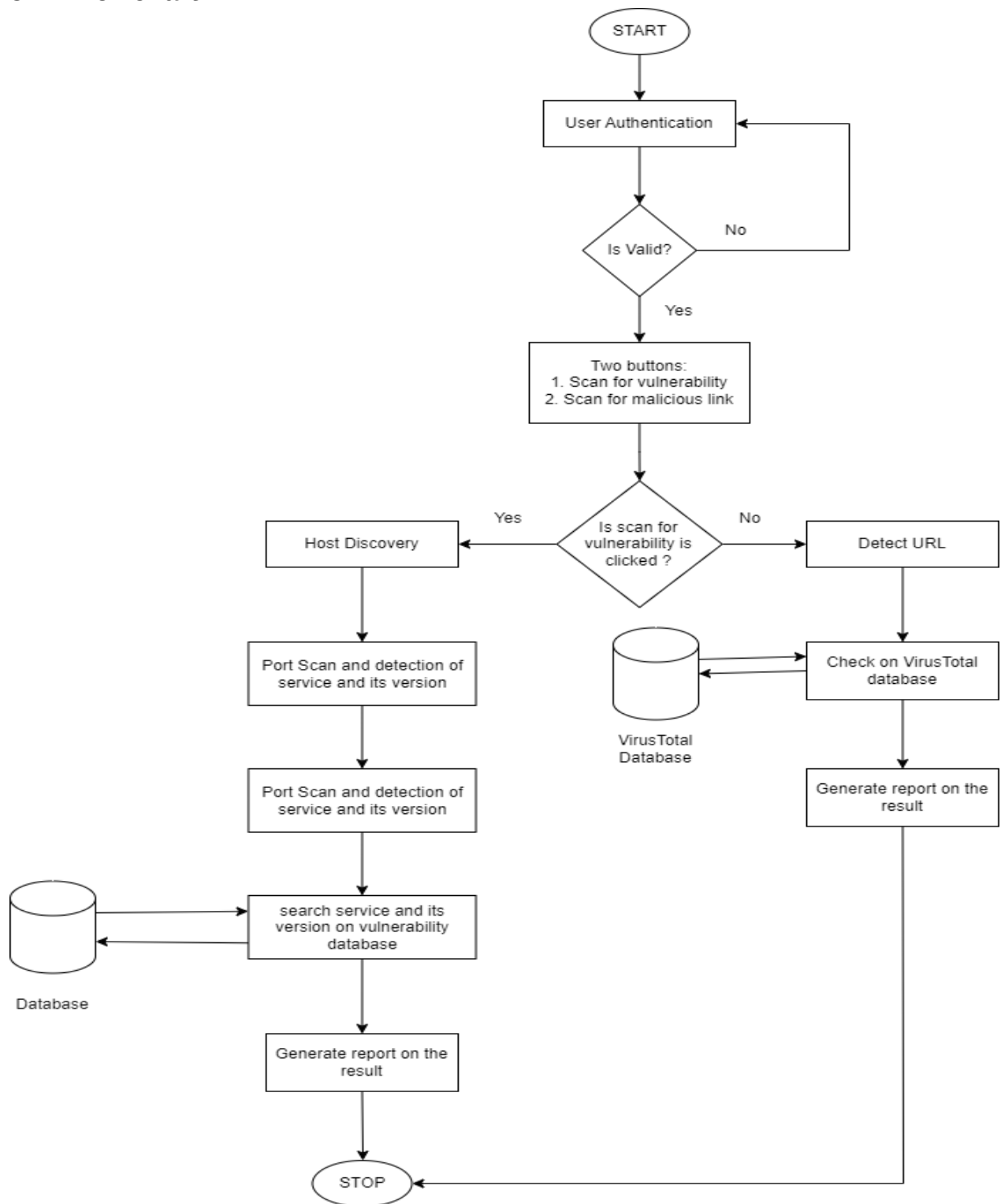


Figure 2: Flowchart of NVA tool

5.2 System architecture diagram

The diagram represents the workflow of a network vulnerability assessment tool. The tool performs a network scan to discover hosts, which are devices connected to the network. Once the hosts are identified, the tool performs port scanning and detects the service and its version. This information is used to find vulnerabilities by searching the service and service version in the vulnerability database, and a report of the results of the scan is generated. The tool also searches for URLs within the network traffic. Each URL is then checked against the VirusTotal database using the VirusTotal API to identify any malicious URLs. The results of the malicious URL detection are compiled, and a separate report on these results is generated.

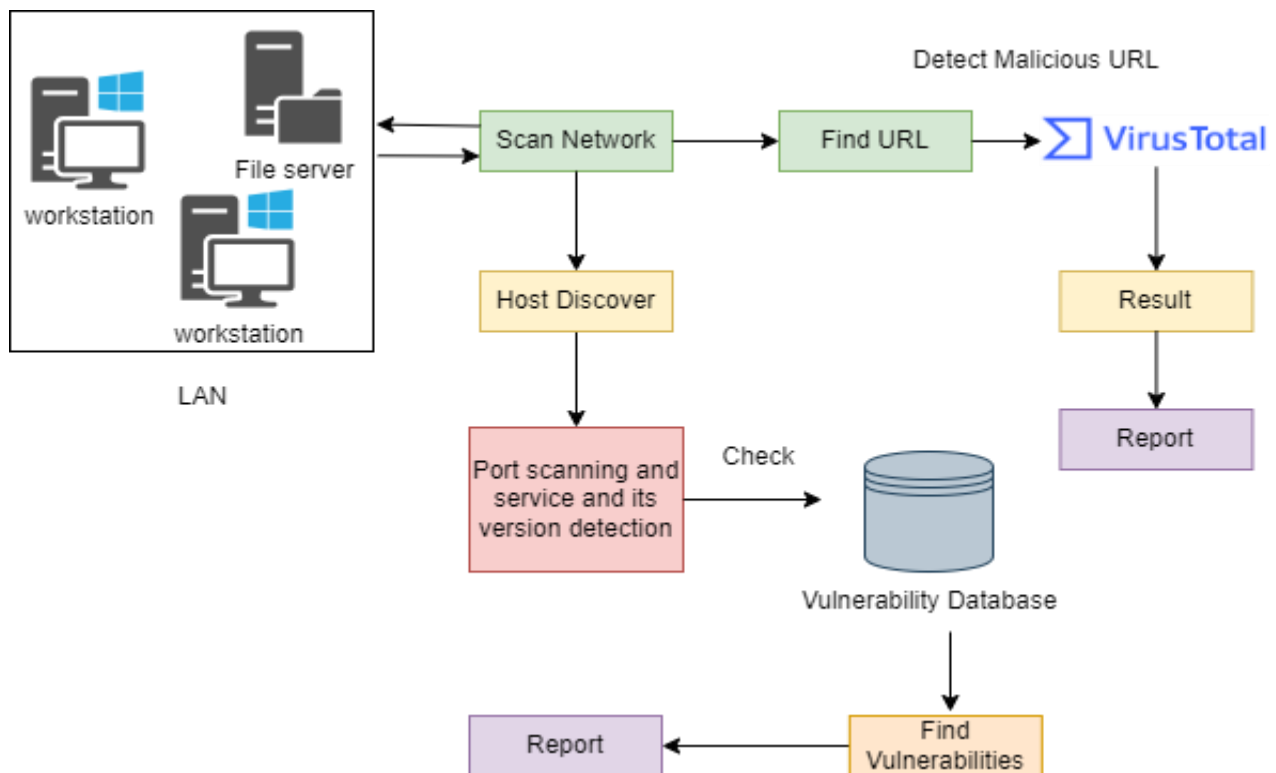


Figure 3: System architecture diagram

6 Resource Requirement

- **Software Requirement**
 - 64-bit OS
 - IDE: vscode
 - Python programming language and library
 - Vulnerability Database
 - Notepad
 - Virustotal API
- **Hardware Requirement**
 - Processor: Intel core i7 and above
 - Processor speed: minimum 2.00 GHz and above
 - RAM: 8-16 GB
 - Disk space: 80 GB

7 Work Breakdown Structure (WBS)

The given work breakdown structure (WBS) represents the different stages and tasks involved in the development of a network vulnerability assessment tool. The project is divided into five main stages: Initiation, Planning, Design, Development, and Finalization. In the initiation stage, a feasibility study is conducted, features are analyzed, requirements are gathered, and research on similar tools and methodologies for the project is carried out. The planning stage involves risk analysis, contingency planning, and the creation of a schedule and tools integration plan. The design stage focuses on designing the user interface and system architecture, as well as designing a prototype of the system. The development stage encompasses system coding and API integration. Lastly, the finalization stage involves conducting system testing, preparing documentation, and assessing project performance.

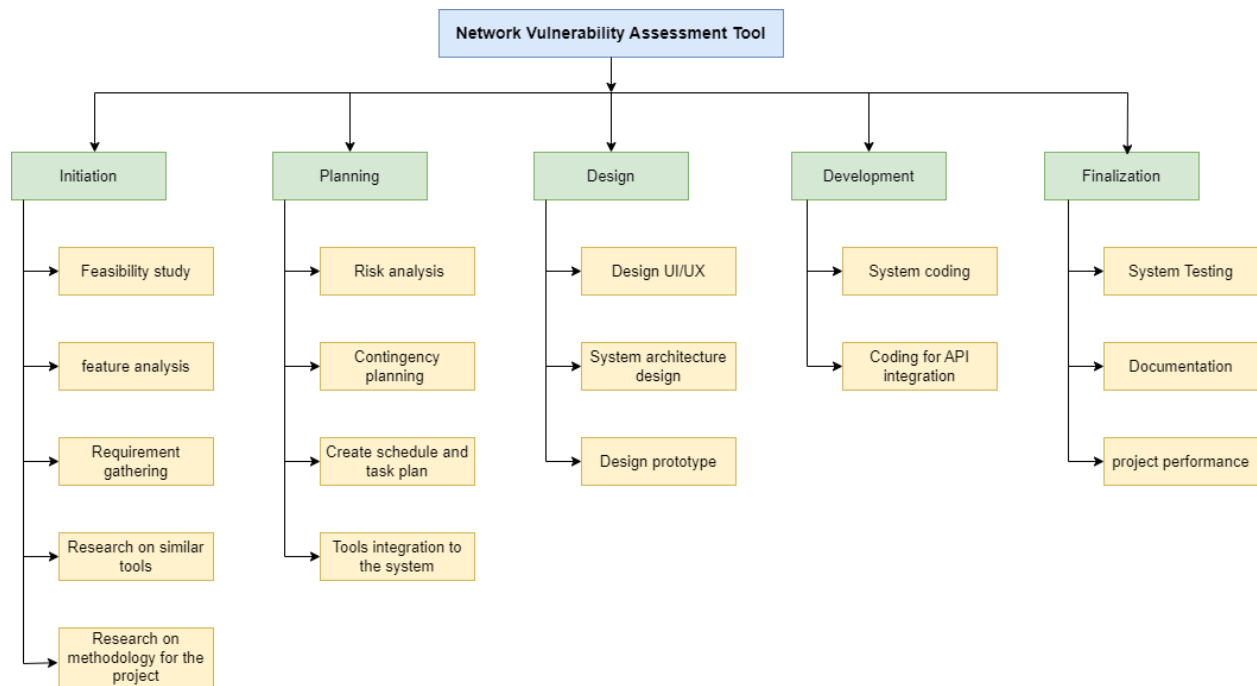


Figure 4: Work breakdown structure (WBS)

8 Milestone

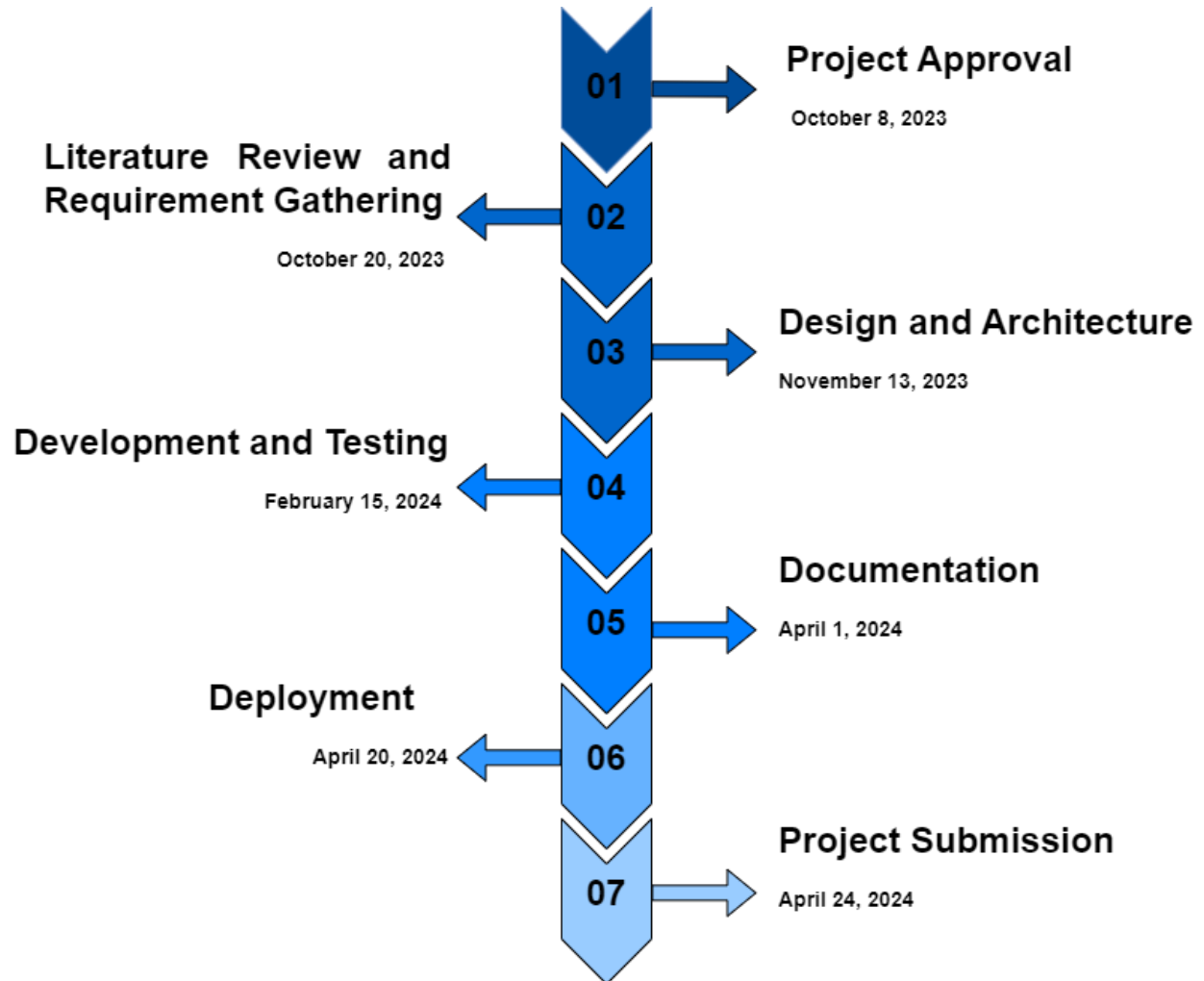


Figure 5: Milestone

9 Project Gantt Chart

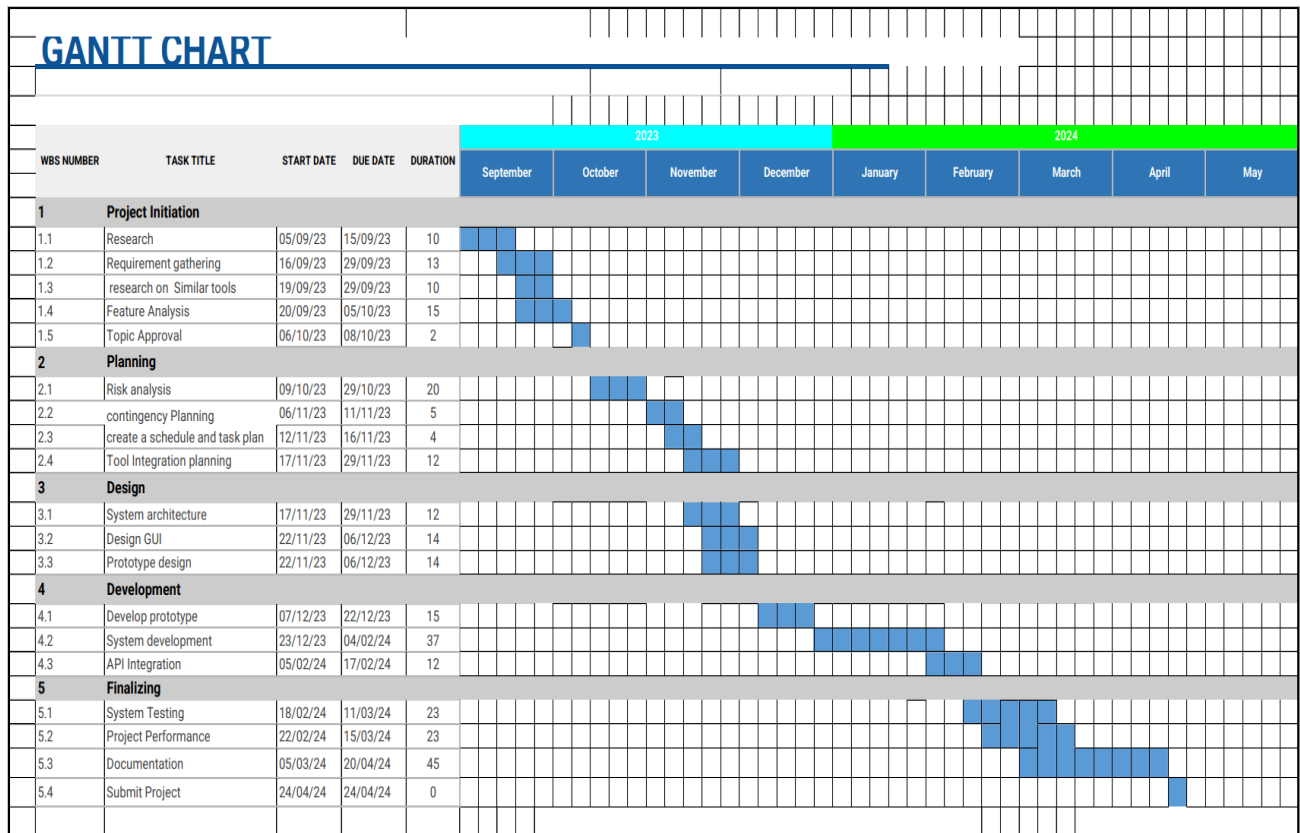


Figure 6: Gantt Chart

10 Conclusion

In conclusion, the Network Vulnerability Assessment Tool is an essential tool for maintaining robust network security. It provides a comprehensive solution to the challenges posed by the ever-evolving landscape of cybersecurity. By incorporating features such as host discovery, open port scanning, service and version detection, vulnerability scanning, and malicious link detection, the tool equips organizations with the means to fortify their network defenses.

The tool's proactive approach to identifying and addressing vulnerabilities helps organizations stay ahead of potential threats, thereby enhancing their cybersecurity posture. Furthermore, the tool's reporting feature provides network administrators and security professionals with actionable insights for mitigation, making it an invaluable asset in the realm of network security. This project is a proactive step towards ensuring the resilience of organizational networks in the face of emerging cyber threats.

11 References

Carter, R. A., Antón, A. . I., Williams, L. A. & Dagnino, A., 2001. Evolving Beyond Requirements Creep: A Risk-Based Evolutionary Prototyping Model.. *Proceedings Fifth IEEE International Symposium on Requirements Engineering*, pp. 1-10.

BtechVibes, 2023. *Prototype Model in Software Engineering*. [Online]
Available at: <https://www.btechvibes.com/2023/06/prototype-model-in-software-engineering.html>
[Accessed 28 11 2023].

Lapada, A., 2019. AIRTOR: A Portable Air Quality Monitor. *Researchgate*, 17(June), p. 12.

Murray, D., 2023. *Rethinking Network Visibility As The Foundation For Security*. [Online]
Available at: <https://www.forbes.com/sites/forbestechcouncil/2023/07/24/rethinking-network-visibility-as-the-foundation-for-security/?sh=7c855c573bf2>
[Accessed 28 11 2023].

Team, W., 2023. *Decoding Network Security in the Digital Age*. [Online]
Available at: <https://www.wrike.com/blog/decoding-network-security/>
[Accessed 28 11 2023].

12 Appendix

12.1 Pros and Cons of the Evolutionary Prototyping Model

The pros of the evolutionary prototyping model are:

- The user engages with the system.
- The system is more likely to meet the user requirements.
- There are fewer chances of errors because all the modules are well-seen.
- It helps to reduce the risk of software projects.

The cons of the evolutionary prototyping model are:

- This model is costly.
- There may be too much variation in requirements.
- This may increase the complexity of the system.

Back to the previous [page](#).